

Project Design Phase

Solution Architecture

Date	15 February 2025
Team ID	
Project Name	AI FAQ Assistant API
Maximum Marks	4 Marks

Solution Architecture:

Solution architecture bridges the gap between business problems and technology solutions. For AI FAQ Assistant API its goals are to:

- Find the best tech solution to the problem of slow, repetitive and unreliable customer support answers.
- Describe the structure, characteristics and behaviour of the API to project stakeholders.
- Define features, development phases and solution requirements.
- Provide specifications according to which the solution is defined, managed and delivered.

Solution Architecture Diagram:

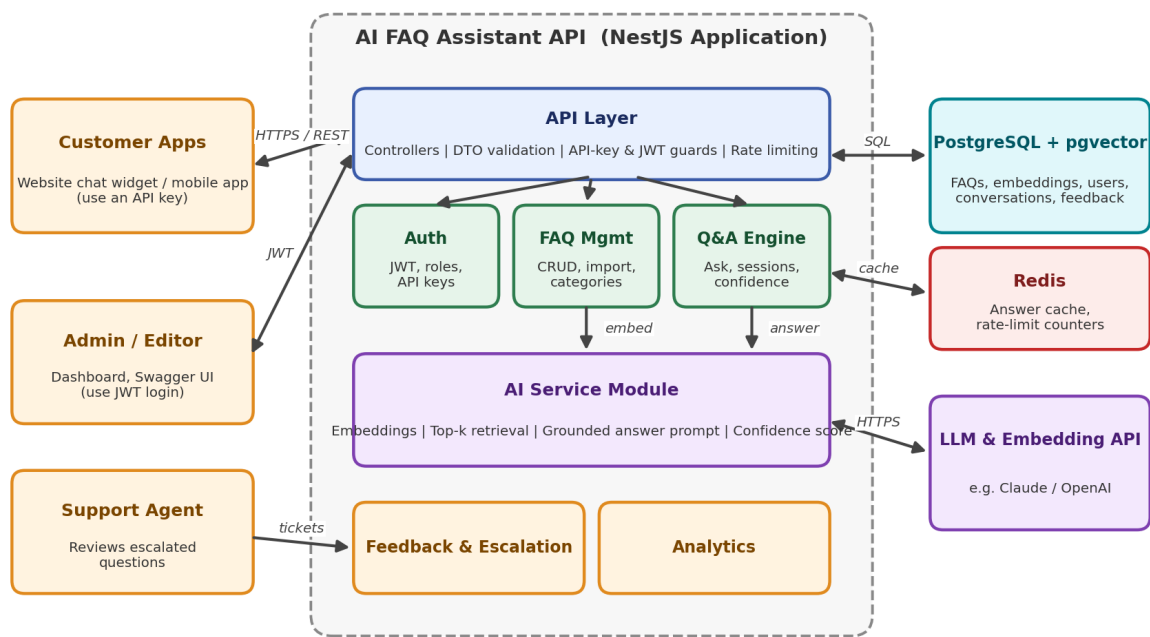


Figure 1: Architecture and data flow of AI FAQ Assistant API

Components:

Component	Responsibility
Customer Apps	A website chat widget or mobile app calls the API over HTTPS with an API key to ask questions and send feedback.
Admin / Editor	Staff manage FAQs and settings using JWT login, a dashboard or Swagger UI.
API Layer	NestJS controllers validate input with DTOs, check JWT or API-key guards and apply rate limiting.
Auth Module	Registration, login, token refresh, roles (Admin, Editor, Agent) and API keys per organisation.
FAQ Management Module	FAQ CRUD, CSV / JSON import, categories, tags and the Draft / Published / Archived workflow. Creates or updates embeddings whenever an FAQ changes.
Q&A Engine	Handles the ask endpoint: cache check, retrieval, confidence decision, conversation sessions and follow-up questions.
AI Service Module	Creates embeddings, finds the top-k similar FAQs, builds the grounded prompt, calls the LLM and calculates a confidence score.
Feedback & Escalation	Stores thumbs up / down, creates tickets for human agents and notifies them by email.
Analytics	Reports top questions, unanswered rate, satisfaction and usage per API key.
PostgreSQL + pgvector	Stores users, FAQs, embedding vectors, conversations, feedback and tickets, with a vector index for similarity search.
Redis	Caches answers to repeated questions and keeps rate-limit counters.
LLM & Embedding API	External model (for example Claude or OpenAI) that creates embeddings and writes the answer text.
Swagger / OpenAPI	Auto-generated interactive documentation for developers.

Data Flow (example: a customer asks a question):

1. The client sends POST /ask with the question, a session ID and its API key.
2. The API layer validates the input, the API-key guard identifies the organisation and the rate limiter checks its quota.
3. The Q&A Engine checks Redis for a cached answer to the same question.
4. On a cache miss, the AI Service creates an embedding of the question and searches pgvector for the closest published FAQs of that organisation.
5. If the best match is below the confidence threshold, the API returns a “no reliable answer” reply and offers to escalate. No answer is invented.
6. Otherwise it builds a prompt containing only the matched FAQ text and asks the LLM to write a short answer.
7. The API returns the answer with source FAQ IDs and a confidence score, saves it to the conversation and caches it.
8. The customer can send feedback or request a human; an escalation creates a ticket and emails an agent.

Technology Stack:

- **Backend framework:** NestJS (Node.js, TypeScript)
- **Database, vector search and cache:** PostgreSQL with pgvector and TypeORM; Redis for caching
- **Security:** JWT, API keys, bcrypt password hashing, role guards, Throttler rate limiting
- **AI:** External LLM and embedding API (Claude / OpenAI) behind a provider-agnostic service
- **Docs, testing and deployment:** Swagger (OpenAPI), Jest for unit and e2e tests, Docker

Reference: <https://docs.nestjs.com/>